SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4307

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%
Dr. V. Parthipan

QUESTIONS: **Total Marks: 30**

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.
Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

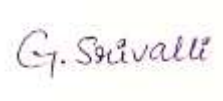
Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I __G. Srivalli (192365014)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

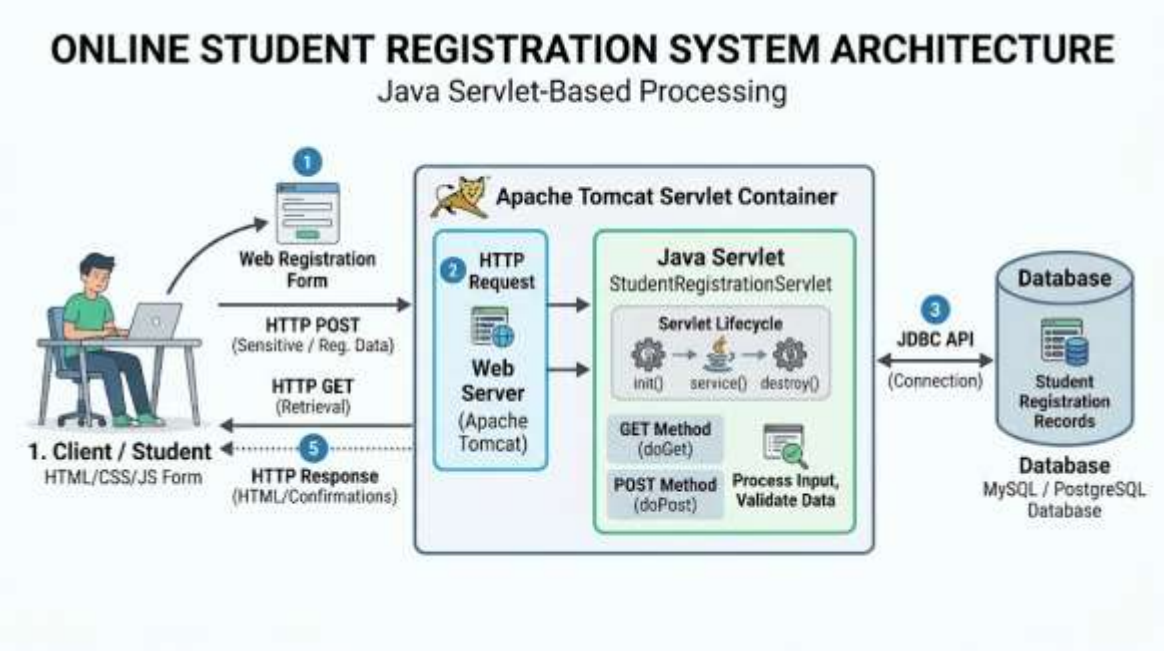
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: Online Student Registration System

1. Architecture of a Servlet-Based Application

A servlet-based web application follows a standard **Three-Tier / Client-Server Web Architecture**:

1. **Client Tier (Presentation Layer):** A web browser renders HTML forms and captures user input.
2. **Server/Application Tier (Servlet Container / Web Tier):** A web server paired with a Servlet Container (such as Apache Tomcat, Eclipse Jetty, or GlassFish) manages the lifecycle of servlets, parses incoming HTTP requests into HttpServletRequest objects, and constructs HttpServletResponse objects.
3. **Data Tier (Backend Layer):** A persistent storage engine (such as MySQL or PostgreSQL) accessed via JDBC.



Communication Flow

1. The student completes an HTML registration form and clicks Submit.
2. The browser packages form fields into an HTTP request (GET query string or POST message body) directed to the servlet's mapped URL pattern (e.g., /register).

3. The Web Server receives the socket connection and delegates it to the Servlet Container.
4. The container allocates or assigns a worker thread from its thread pool, instantiates/reuses the target servlet, and invokes service(HttpServletRequest request, HttpServletResponse response).
5. The service() method inspects the HTTP method token and routes execution to doGet() or doPost().
6. The servlet extracts parameters, applies business logic or database operations, generates dynamic HTML/JSON, flushes the output stream, and terminates the thread execution.

2. Design and Implementation: HTML Form & Registration Servlet

HTML Registration Form (register.html)

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Student Registration System</title>
</head>
<body>
    <h2>Online Student Registration</h2>
    <!-- Uses POST for secure submission of personal details -->
    <form action="StudentRegistrationServlet" method="POST">
        <label for="studentId">Student ID:</label><br>
        <input type="text" id="studentId" name="studentId" required><br><br>
        <label for="studentName">Full Name:</label><br>
        <input type="text" id="studentName" name="studentName" required><br><br>
        <label for="email">Email Address:</label><br>
        <input type="email" id="email" name="email" required><br><br>
        <label for="department">Department:</label><br>
        <select id="department" name="department">
            <option value="Computer Science">Computer Science</option>
            <option value="Information Technology">Information Technology</option>
            <option value="Electronics">Electronics</option>
```

```
        </select><br><br>

        <label for="password">Portal Password:</label><br>
        <input type="password" id="password" name="password" required><br><br>
        <input type="submit" value="Register Student">
    </form>
</body>
</html>
```

Java Servlet Implementation (StudentRegistrationServlet.java)

Java

```
package com.simats.registration;
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
@WebServlet("/StudentRegistrationServlet")
public class StudentRegistrationServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    @Override
    public void init() throws ServletException {
        // One-time initialization logic (e.g., logging or resource allocation)
        System.out.println("StudentRegistrationServlet initialized.");
    }
    // Handles query/read requests (e.g., checking registration status or instructions)
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            String queryId = request.getParameter("studentId");
```

```
            out.println("<!DOCTYPE html><html><head><title>Registration
Info</title></head><body>");
            if (queryId != null && !queryId.trim().isEmpty()) {
                out.println("<h3>Checking status for Student ID: " + queryId + "</h3>");
                out.println("<p>Status: Verification Pending.</p>");
            } else {
                out.println("<h3>Student Registration Portal</h3>");
                out.println("<p>Please submit the registration form using the official
portal.</p>");
            }
            out.println("</body></html>");
        }
    }
    // Handles form data submission (creates/updates registration state)
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        // Extracting form fields
        String studentId = request.getParameter("studentId");
        String studentName = request.getParameter("studentName");
        String email = request.getParameter("email");
        String department = request.getParameter("department");
        String password = request.getParameter("password"); // Sensitive field
        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE html><html><head><title>Registration
Confirmation</title></head><body>");
            // Server-side validation
            if (studentId == null || studentName == null || password == null ||
                studentId.trim().isEmpty() || studentName.trim().isEmpty() ||
password.isEmpty()) {
                out.println("<h3 style='color:red;'>Error: Missing mandatory registration
fields.</h3>");
```

```
            out.println("<a href='register.html'>Back to Form</a>");
        } else {
            // In production, delegate to a DAO layer to persist data securely
            out.println("<h2 style='color:green;'>Registration Successful!</h2>");
            out.println("<p><strong>Student ID:</strong> " + studentId + "</p>");
            out.println("<p><strong>Name:</strong> " + studentName + "</p>");
            out.println("<p><strong>Email:</strong> " + email + "</p>");
            out.println("<p><strong>Department:</strong> " + department + "</p>");
            out.println("<p><em>Password securely received and hashed for storage.</em></p>");
        }
        out.println("</body></html>");
    }
  }
  @Override
  public void destroy() {
    // Cleanup resources before servlet is removed by the container
    System.out.println("StudentRegistrationServlet destroyed.");
  }
}
```

3. Servlet Life Cycle

The life cycle of a servlet is managed entirely by the servlet container through three core methods:

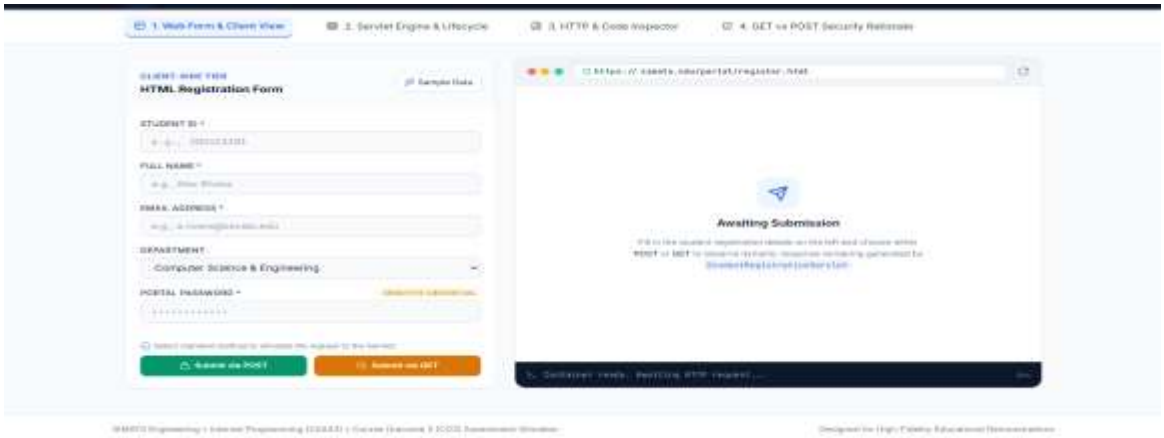
1. **init(ServletConfig config):**
    - Invoked once when the servlet is first loaded into memory (either upon the first request or at container startup if <load-on-startup> is configured).
    - Used for heavy one-time initialization, such as opening shared resources or reading configuration settings.
2. **service(ServletRequest req, ServletResponse res):**
    - Invoked for every incoming client request.
    - Runs in a separate worker thread within the container's thread pool.
    - Dispatches execution to HTTP-specific handlers (doGet, doPost, doPut, doDelete) in HttpServlet.

3. **destroy():**
    - Invoked once before the servlet instance is deallocated from memory by the container (e.g., during application undeploy or server shutdown).
    - Used to release open database connections, stop background threads, and flush buffers.

4. Method Justification for Handling Sensitive Data: POST vs. GET

Evaluation Parameter	**HTTP GET**	**HTTP POST**
Data Placement	Appended to the URL in the query string (?key=val)	Embedded within the HTTP Request Message Body
Visibility / Security	Highly visible in address bar, browser history, proxy logs, and Referer headers	Hidden from URLs, logs, and browser history
Payload Size Limit	Severely restricted by URI length limits (~2048 characters depending on browser/server)	Virtually unlimited (configurable by server)
Idempotency & Caching	Idempotent, safe, and cached by default	Non-idempotent; not cached by default

Verdict: HTTP POST is the required method for student registration. Because the form collects sensitive attributes—specifically passwords, personal email addresses, and identification numbers—transmitting them via GET would expose raw credentials in browser histories, web server access logs, and intermediate proxy caches. POST, especially when wrapped in TLS/HTTPS, encapsulates the payload inside an encrypted request body.



Question 2: Session Management System

1. How Session Tracking Works in Web Applications

HTTP is a **stateless protocol**: every request-response cycle is completely independent, and the server retains no built-in memory of prior requests from the same client.

To maintain state across multiple page interactions (e.g., keeping a user logged in), web applications use **Session Tracking**:

1. When a client authenticates successfully, the server creates a unique session record in its memory or persistent cache.
2. The server generates a globally unique identifier (e.g., JSESSIONID).
3. This identifier is sent back to the client, typically via a Set-Cookie HTTP header.
4. On subsequent requests, the client automatically attaches this identifier via the Cookie request header.
5. The container matches the incoming token against its active session table to retrieve the associated user state.

2. Design and Implementation: Login, Dashboard, and Logout

A. Authentication Servlet (LoginServlet.java)

Java

```
package com.simats.session;
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
```

```
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {
   private static final long serialVersionUID = 1L;
   @Override
   protected void doPost(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {
      String username = request.getParameter("username");
      String role = request.getParameter("role"); // e.g., "Student" or "Faculty"
      // Mock authentication check
      if (username != null && !username.trim().isEmpty()) {
         // 1. Establish HttpSession on the server side
         HttpSession session = request.getSession(true); // Create new session
         session.setAttribute("currentUser", username);
         session.setAttribute("userRole", role != null ? role : "Student");
         session.setMaxInactiveInterval(15 * 60); // 15-minute timeout
         // 2. Establish a client-side tracking cookie (e.g., theme or visit count)
         Cookie userPreferenceCookie = new Cookie("userTheme", "Dark");
         userPreferenceCookie.setMaxAge(7 * 24 * 60 * 60); // Persist for 7 days
         userPreferenceCookie.setHttpOnly(true);          // Prevent XSS access
         userPreferenceCookie.setPath("/");               // Global scope
         response.addCookie(userPreferenceCookie);
         // Redirect to protected dashboard
         response.sendRedirect("DashboardServlet");
      } else {
         response.setContentType("text/html");
         PrintWriter out = response.getWriter();
         out.println("<p style='color:red;'>Invalid credentials. <a href='login.html'>Try again</a></p>");
      }
```

```
    }
}
```

B. Protected Resource Servlet (DashboardServlet.java)

Java

```
package com.simats.session;
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
@WebServlet("/DashboardServlet")
public class DashboardServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        // Validate session without creating a new one
        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("currentUser") == null) {
            // Unauthorized access; redirect to login
            response.sendRedirect("login.html");
            return;
        }
        String username = (String) session.getAttribute("currentUser");
        String role = (String) session.getAttribute("userRole");
        // Read client-side cookie
        String theme = "Default";
        Cookie[] cookies = request.getCookies();
        if (cookies != null) {
```

```
            for (Cookie c : cookies) {
                if ("userTheme".equals(c.getName())) {
                    theme = c.getValue();
                    break;
                }
            }
        }

        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE
html><html><head><title>Dashboard</title></head><body>");
            out.println("<h1>Welcome to your Dashboard, " + username + "!</h1>");
            out.println("<p><strong>Assigned Role:</strong> " + role + "</p>");
            out.println("<p><strong>Session ID:</strong> " + session.getId() + "</p>");
            out.println("<p><strong>Active Theme (from Cookie):</strong> " + theme +
"</p>");
            out.println("<br><a href='LogoutServlet'>Log Out</a>");
            out.println("</body></html>");
        }
    }
}
```

C. Session Invalidation Servlet (LogoutServlet.java)

Java

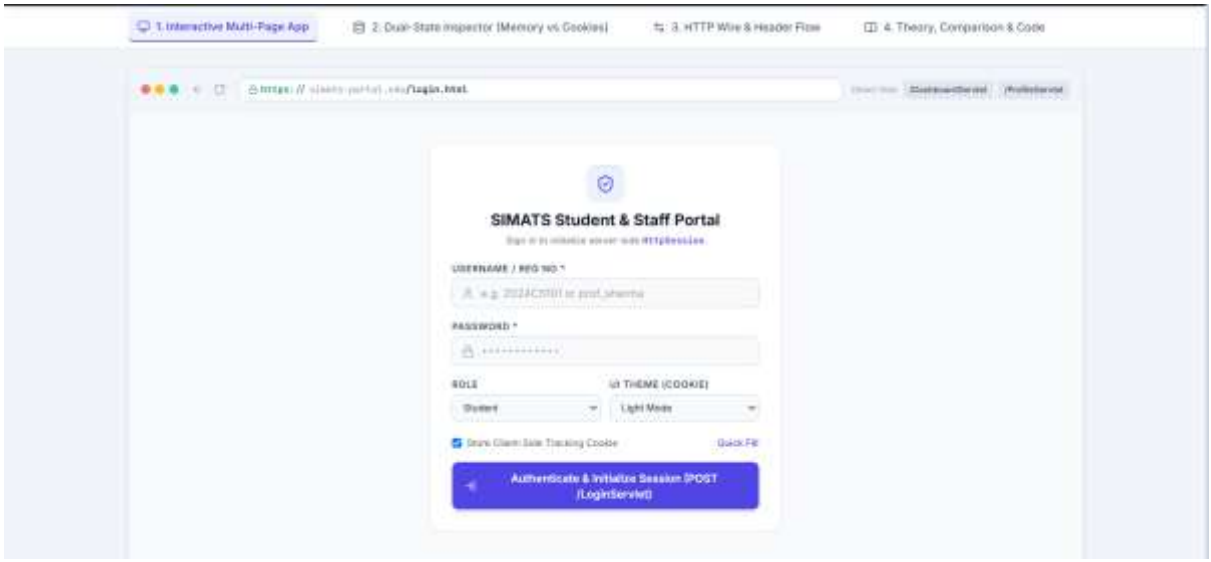
```
package com.simats.session;
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
@WebServlet("/LogoutServlet")
public class LogoutServlet extends HttpServlet {
```

```
    private static final long serialVersionUID = 1L;
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // 1. Invalidate Server-Side Session
        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();
        }
        // 2. Clear Client-Side Tracking Cookies
        Cookie killCookie = new Cookie("userTheme", "");
        killCookie.setMaxAge(0); // Immediately delete cookie from browser
        killCookie.setPath("/");
        response.addCookie(killCookie);
        // Redirect back to login page
        response.sendRedirect("login.html");
    }
}
```

3. Comparison: Session-Based Tracking (HttpSession) vs. Cookies

Feature	HttpSession (Server-Side)	Cookies (Client-Side)
Storage Location	Server memory / distributed cache (Redis, DB)	Client machine (browser cache / text file)
Data Capacity	Effectively unlimited (bounded by server RAM)	Limited to 4 KB per cookie, max ~20-50 per domain
Data Types Supported	Any arbitrary Java Object (Object, List, Map)	Text/Strings only
Security	High. Data stays on server; client holds only the token	Low to Moderate. Exposed to tampering unless encrypted and protected with HttpOnly and Secure flags

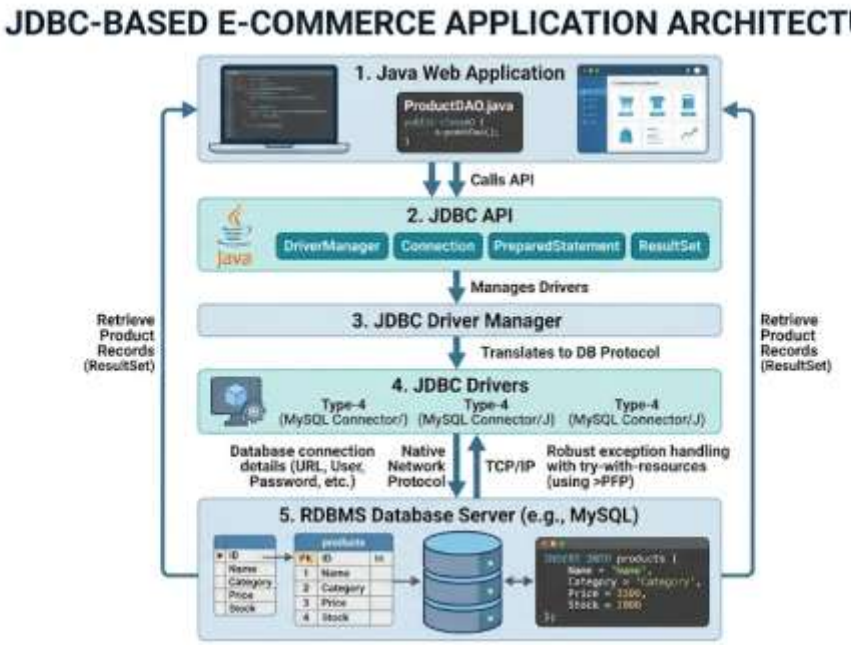
Network Overhead	Low. Only the session ID transmits across requests	Higher. Cookie payload is sent in full on every HTTP request header
Lifecycle / Expiry	Managed by server inactivity timeout (session.setMaxInactiveInterval())	Managed by explicit expiry date (cookie.setMaxAge()); persists across browser restarts
Primary Use Cases	Authentication states, shopping carts, role permissions	Tracking preferences, analytics, "Remember Me" tokens



Question 3: JDBC-Based E-Commerce Application

1. JDBC Architecture

Java Database Connectivity (JDBC) is an industry-standard Java API that enables Java applications to interact with relational database management systems (RDBMS) in a database-independent manner.



Architectural Components

1. **Application Layer:** Calls standard JDBC interfaces to execute queries and process results.
2. **JDBC API:** Set of built-in Java abstractions (Connection, Statement, PreparedStatement, ResultSet, CallableStatement).
3. **JDBC Driver Manager:** Dispatches database-specific connection requests to the registered database driver.
4. **JDBC Drivers:** Vendor-supplied client libraries implementing the standard JDBC interfaces. In modern production, **Type 4 (Pure Java Native-Protocol Driver)** is standard because it converts JDBC calls directly into vendor-specific database protocols without requiring native client libraries.

2. Standard Steps in Database Connectivity

Connecting and executing queries via JDBC follows six distinct steps:

1. **Load/Register Driver:** Register the vendor's driver class (handled automatically via SPI in modern JDBC 4.0+ via META-INF/services/java.sql.Driver, or manually loaded using Class.forName("com.mysql.cj.jdbc.Driver")).
2. **Establish Connection:** Obtain a physical socket connection using DriverManager.getConnection(url, user, password) or a connection pool (DataSource).

3. **Create Statement:** Formulate parameterized SQL commands using PreparedStatement to prevent SQL Injection and pre-compile execution plans.
4. **Execute Query / Update:** Execute the command using executeQuery() (for SELECT) or executeUpdate() (for INSERT, UPDATE, DELETE).
5. **Process Results:** Iterate over the returned ResultSet using a cursor-driven loop (while(rs.next())).
6. **Close Resources:** Release database locks and memory handles via Java's try-with-resources block (ensuring AutoCloseable execution in reverse order of creation: ResultSet → PreparedStatement → Connection).

3. Implementation: Product Data Operations (ProductDAO.java)

Database Schema

SQL

```
CREATE DATABASE ecommerce_db;
USE ecommerce_db;
CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price DOUBLE NOT NULL,
    stock_quantity INT NOT NULL
);
```

Complete Data Access Object Implementation

Java

```
package com.simats.ecommerce;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
public class ProductDAO {
    // Database connection coordinates
```

```
    private static final String DB_URL = "jdbc:mysql://localhost:3306/ecommerce_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "password";
    // Static block to guarantee explicit driver registration
    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            System.err.println("Failed to load MySQL JDBC Driver: " + e.getMessage()); }
    }
    // Helper method to establish connection
    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
    }
    /**
     * Inserts a product record into the database using a parameterized PreparedStatement.
     */
    public boolean insertProduct(Product product) {
        String insertSQL = "INSERT INTO products (product_name, category, price, stock_quantity) VALUES (?, ?, ?, ?)";
        // Try-with-resources automatically closes Connection and PreparedStatement
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
            pstmt.setString(1, product.getName());
            pstmt.setString(2, product.getCategory());
            pstmt.setDouble(3, product.getPrice());
            pstmt.setInt(4, product.getStockQuantity());
            int rowsAffected = pstmt.executeUpdate();
            return rowsAffected > 0;
        } catch (SQLException e) {
            // Comprehensive SQL Exception logging
```

```
            System.err.println("Database Error during Product Insertion!");
            System.err.println("Message: " + e.getMessage());
            System.err.println("SQLState: " + e.getSQLState());
            System.err.println("Error Code: " + e.getErrorCode());
            e.printStackTrace();
            return false; }
    }
    /**
     * Retrieves all products matching a specific category from the database.
     */
    public List<Product> getProductsByCategory(String category) {
        List<Product> products = new ArrayList<>();
        String selectSQL = "SELECT product_id, product_name, category, price,
stock_quantity FROM products WHERE category = ?";
        // Try-with-resources handles Connection, PreparedStatement, and ResultSet
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(selectSQL)) {
            pstmt.setString(1, category);
            try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    Product prod = new Product();
                    prod.setId(rs.getInt("product_id"));
                    prod.setName(rs.getString("product_name"));
                    prod.setCategory(rs.getString("category"));
                    prod.setPrice(rs.getDouble("price"));
                    prod.setStockQuantity(rs.getInt("stock_quantity"));
                    products.add(prod); }
            }
        } catch (SQLException e) {
            System.err.println("Database Error during Product Retrieval!");
            System.err.println("Message: " + e.getMessage());
            System.err.println("SQLState: " + e.getSQLState());
            e.printStackTrace();}
        return products; }
```

```
    // Standalone verification runner
    public static void main(String[] args) {
        ProductDAO dao = new ProductDAO()
        // 1. Insert product
        Product laptop = new Product("UltraBook Pro", "Electronics", 1299.99, 15);
        boolean isInserted = dao.insertProduct(laptop);
        System.out.println("Insert status: " + (isInserted ? "SUCCESS" : "FAILURE"));
        // 2. Retrieve products
        List<Product> electronics = dao.getProductsByCategory("Electronics");
        System.out.println("\nRetrieved Products under 'Electronics':");
        for (Product p : electronics) {
            System.out.println("- [" + p.getId() + "] " + p.getName() + " | $" + p.getPrice() +
" | Stock: " + p.getStockQuantity()); }
    }
}
```

Model Class (Product.java)

Java

```
package com.simats.ecommerce;
public class Product {
    private int id;
    private String name;
    private String category;
    private double price;
    private int stockQuantity;
    public Product() {}
    public Product(String name, String category, double price, int stockQuantity) {
        this.name = name;
        this.category = category;
        this.price = price;
        this.stockQuantity = stockQuantity; }
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
```

```
    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public int getStockQuantity() { return stockQuantity; }
    public void setStockQuantity(int stockQuantity) { this.stockQuantity = stockQuantity;
}
}
```

4. Robust Exception Handling and Resource Management

- **Preventing Leaks with AutoCloseable:** Opening database connections consumes physical OS sockets and database cursor slots. By enclosing all Connection, PreparedStatement, and ResultSet instantiations in a **try-with-resources** block, resources are deterministically closed in reverse order of initialization, even if an unhandled runtime error or query timeout occurs.
- **SQL Injection Defense:** All queries use PreparedStatement with typed parameter placeholders (?). Values are bound safely via parameterized drivers, neutralizing SQL injection vectors.
- **Granular Diagnostic Logging:** Catching SQLException inspects e.getMessage(), e.getSQLState(), and e.getErrorCode(), distinguishing connection failures (e.g., network drop, bad credentials) from syntax errors or database constraint violations (e.g., duplicate unique keys).

